When the weakest model sees the threat: an explainable ensemble learning system for detecting network attacks

Céline Minh, Kevin Vermeulen, Cédric Lefebvre, Philippe Owezarski, William Ritchie

When the weakest model sees the threat: an explainable ensemble learning system for detecting network attacks

Céline Minh[a,b,*], Kevin Vermeulen[b], Cédric Lefebvre[a], Philippe Owezarski[b], William Ritchie[a]

[a] *Custocy, Toulouse, France*
[b] *LAAS-CNRS, Université de Toulouse, CNRS, Toulouse, France*

Abstract

The growing importance of network security is driven by two major challenges. One is the exponential increase in network traffic, which exceeds human processing capabilities. The other is the rising frequency and sophistication of attacks, which demand advanced, intelligent analysis. To take critical actions—such as blocking suspicious IP addresses—security analysts must understand why intrusion detection systems raise alarms. This highlights the limitations of relying on machine learning models with opaque decision-making processes, often referred to as "black boxes," whose lack of interpretability poses challenges for justifying security actions. Consequently, this paper emphasizes the need for explainable machine learning solutions tailored to network security. To detect new attacks effectively, we adopt a behavioral approach that analyzes short time windows of aggregated traffic to identify abnormal patterns, using various unsupervised machine learning detectors. These detectors often yield complementary results: they may disagree on specific detections, and in some cases, a generally less effective model—the weakest—can uniquely identify an attack. This observation motivates an ensemble approach that integrates the strengths of diverse models. Our approach combines three key contributions: a stacking-based ensemble learning strategy that improves detection by incorporating minority reports, going beyond majority voting; a visual representation technique

*Corresponding author

Email addresses: cminh@custocy.com (Céline Minh), kevin.vermeulen@laas.fr (Kevin Vermeulen), clefebvre@custocy.com (Cédric Lefebvre), owe@laas.fr (Philippe Owezarski), writchie@custocy.com (William Ritchie)

that transforms anomaly scores into heatmaps, making outputs more interpretable to analysts; and the use of convolutional neural networks to process these heatmaps, simulating human reasoning and improving pattern recognition. Altogether, we develop and evaluate an unsupervised, explainable system capable of detecting even previously unknown network attacks by combining ensemble learning with visual interpretability.

Keywords: network security, unsupervised anomaly detection, explainable AI, ensemble learning, CNN

---

1. Introduction

Modern enterprise networks generate a constant stream of high-volume, heterogeneous traffic—from remote work connections and cloud-based services to IoT communications and data exchanges between different corporate sites. This complex and dynamic environment creates fertile ground for attackers seeking to exploit blind spots in detection and response. In this context, securing networks has become increasingly challenging due to two converging trends: the exponential growth in traffic and the rising sophistication of cyberattacks.

The first challenge is the unprecedented rise in network traffic volumes, driven in part by the widespread adoption of remote work, which has significantly altered network traffic patterns and introduced new vulnerabilities. Today, the sheer volume of network data far exceeds human processing capabilities, making automated tools like Network Intrusion Detection Systems (NIDS) indispensable for helping security analysts detect potential attacks.

Faced with this surge in traffic, security analysts are required to review a growing number of alerts to identify and respond to actual threats. This is compounded by the complexity and volume of these alerts, which can lead to information overload and increase the risk of critical errors, as well as analyst fatigue. Furthermore, large volumes of unclear alerts may result in analysts giving less attention to important signals, thus undermining the effectiveness of NIDS in practice [3, 15].

The second challenge stems from the emergence of more advanced and evasive cyberattacks. The economic and strategic incentives behind these attacks motivate attackers to develop advanced methods to bypass existing security systems. For instance, advanced persistent threats (APTs) can remain undetected for months or even years within a network [23]. Additionally, the

rise of generative AI, marked by tools like ChatGPT, has simplified access to adversarial techniques, enabling even less experienced attackers to launch effective cyberattacks by training AI models to deceive existing systems.

Traditionally, rule-based systems, such as Snort and Suricata, have been the primary tools to address these challenges [31]. However, this approach assumes that all forms of attacks can be accurately characterized—a notion that is becoming increasingly unrealistic. The vast diversity and ingenuity of current threats make it difficult to exhaustively define attack signatures. Furthermore, the frequency with which new attacks appear quickly renders existing signature databases obsolete, reducing the effectiveness of rule-based detection in real time.

To address these challenges, inductive approaches that dynamically adapt to new attacks are becoming essential. Recent advancements in machine learning have led to the development of models that demonstrate impressive detection capabilities in lab environments. However, these improvements often come at the cost of increased model complexity, leading to the rise of so-called "black-box" systems. This introduces a critical issue regarding explainability. Without clear insights into the reasons behind the alerts, security analysts struggle to trust and effectively act on them, compromising incident response. Furthermore, this black-box nature often forces developers into blind correction efforts, tweaking hyperparameters and retraining models without understanding the root cause of errors.

While explainability is essential for informed decision-making in Security Operations Centers (SOCs), achieving robust detection performance without compromising transparency requires advanced strategies. In search of this balance, our work explores unsupervised learning techniques combined with an innovative stacking-based ensemble approach. A key finding of our research reveals that certain anomaly detectors, while less accurate overall, can successfully identify attacks that more generally effective detectors miss. This observation highlights the limitations of common ensemble learning methods — techniques that combine multiple models to improve prediction — such as weighted voting, which often overlooks "minority reports"—cases where a small subset of detectors signals an attack undetected by the majority.

To address this issue, we introduce a new stacking strategy that emphasizes the complementary strengths of multiple anomaly detectors by valuing minority detections when they are relevant. This refined analysis not only enhances the synergy between the models but also contributes to the system's explainability by showcasing the unique capacities of each detector within

the ensemble.

Another core innovation of our approach lies in the design of intuitive visual representations of network anomalies. These concise "heatmaps" provide analysts with a clear overview of network states, facilitating both understanding and swift response. The heatmaps are further processed by Convolutional Neural Networks (CNNs), which, commonly used in computer vision, here simulate the way a security analyst might visually inspect the health of a network. This method, inspired by human analytical techniques, strengthens the system's overall transparency and allows each detection result to be accompanied by an intuitive, data-backed report.

This work focuses on the design of a machine learning (ML)-based Network Intrusion Detection System (NIDS) intended for integration into Custocy's network detection and response (NDR) solution [1], aimed at enterprises. The proposed system addresses concrete requirements identified during the development of Custocy's solution, including robust unsupervised detection capabilities and integrated interpretability to support operational use.

In this paper, we propose an explainable-by-design NIDS that leverages the strengths of various anomaly detectors and CNN-based visual interpretation. Our main contributions are:

1. A new stacking-based ensemble learning strategy that improves detection accuracy by incorporating minority reports, effectively addressing cases where simpler majority-voting methods fall short.
2. A visual representation approach that transforms anomaly scores into heatmaps, enhancing interpretability for security analysts.
3. An integration of CNNs to process visual data, simulating human analysis to identify complex attack patterns while maintaining transparency in the detection process.

2. Related Work

Our research explores mechanisms to reconstruct attack patterns using multiple unsupervised learning detection techniques. To achieve this, we address key challenges in the fields of (1) ensemble learning, where combining diverse models can improve detection coverage and accuracy; (2) event correlation and attack pattern recognition, which focuses on reconstructing complex attack sequences from individual anomaly detections; and (3)

explainable AI, ensuring that our system's outputs are interpretable and actionable for security analysts.

2.1. Ensemble Learning for Enhanced Anomaly Detection

Ensemble learning techniques, which integrate multiple models, offer a variety of general-purpose approaches aiming to enhance robustness and address the limitations of individual algorithms. Techniques such as bagging [6], boosting [11, 12], and stacking [34] exemplify this, each leveraging the complementary strengths of diverse base learners to improve detection accuracy. Our focus, however, is on stacking, as it allows us to address the "minority report" challenge by capturing and integrating detections from base learners that may identify specific attacks overlooked by others.

In the context of network security, Vanerio and Casas [30] demonstrated the effectiveness of weighted majority voting, where base learners' weights depend on accuracy, improving detection in some scenarios. However, this approach may overlook cases where models with lower overall accuracy excel in specific attack detections, highlighting the need for ensemble methods that better capture this "minority report" effect.

Mirsky et al. [24] proposed Kitsune, an ensemble of autoencoders for anomaly detection. In this system, anomaly scores computed by the ensemble are further processed by a secondary autoencoder, referred to as a meta-learner. While effective, this approach is limited to homogeneous models and requires significant training on normal traffic data, potentially reducing its adaptability to diverse attack types.

We extend prior work by applying stacking with a CNN meta-learner to integrate outputs from heterogeneous detectors, specifically addressing the "minority report" issue neglected by majority-based methods.

2.2. Event Correlation and Attack Pattern Recognition

Sophisticated attacks, such as multi-stage attacks, present significant challenges for intrusion detection, as they often unfold over long periods and consist of sequences of seemingly benign events that, when correlated, reveal coordinated malicious intent. Models such as the Cyber Kill Chain [16], Mandiant's APT lifecycle [22], and the MITRE ATT&CK framework [2] conceptualize attacks as sequences of stages or tactics. While these frameworks provide valuable insights, they rely heavily on predefined stages, limiting flexibility in identifying new or evolving attack patterns.

Some studies still incorporate these lifecycle concepts directly into their model development: Zhou et al.[36] utilized LSTM networks to capture long-term dependencies in alert sequences for multi-stage attack detection, while Ghafir et al.[13] applied a Hidden Markov Model to detect and predict APT stages based on past behavior. However, both methods are constrained by training on known attack lifecycles, reducing adaptability to unknown threats. Wang et al. [32] proposed an image-based method, converting raw traffic data into visual formats analyzed by a CNN to detect attack patterns, though this approach may lack semantic clarity for human analysts.

Our system goes beyond prior work by correlating anomalies across time in an unsupervised manner, without relying on predefined attack lifecycles. The CNN meta-learner processes visual representations specifically designed to be both informative for models and readable by analysts, improving detection accuracy while supporting interpretation and rapid response.

2.3. Explainable AI for Intrusion Detection

Explainability remains essential for effective decision-making in network security, yet traditional methods often fall short in this domain. Techniques like LIME [27] and SHAP [21] provide general-purpose explanations, but they struggle to handle the dependencies among network traffic features, often resulting in incomplete insights [33]. Such methods, while model-agnostic, do not consistently provide the contextual clarity required for security analysts to interpret alerts accurately or act on them swiftly.

Recent research has explored more tailored solutions for explainability in deep learning-based NIDS. Wei et al.[33] proposed data-driven explanations that incorporate historical inputs, enhancing interpretation by generating feature importance insights used to define defensive rules. Han et al.[14] designed an interpretation system for unsupervised DL-based NIDS, identifying key features involved in each detection and offering contextual explanations to aid analyst understanding.

Our system is explainable-by-design, with a modular structure that mirrors how analysts decompose, interpret, and make sense of network signals. Each component contributes to producing intermediate representations that support step-by-step, transparent reasoning and are directly usable by analysts.

3. System Design

3.1. System Overview

We developed an AI-based NIDS that employs a set of unsupervised base learners (UL) to detect network anomalies. These base learners are orchestrated by a meta-learner, a CNN, to detect attack patterns (Figure 1). A key aspect of our design is combining different anomaly detection algorithms, applying them to the same data. In the rest of this paper, we refer to each algorithm-instance as a "model", since each represents a distinct detection method applied independently to the data. By focusing solely on the inputs and outputs of base learners, our approach characterizes their behavior independently of specific algorithm definitions. The overall system is developed as a research prototype, aiming to validate the feasibility of interpretable ensemble-based detection.

For model inputs, we required a semantically meaningful representation for effective attack detection (Section 3.2). Inspired by UNADA [8, 9], we aggregated network flows by source and destination IP addresses, defining statistical features for each aggregate (Table 1).

Each base learner assigns an anomaly score to an aggregate by applying the same algorithm across different feature subspaces. The outputs are then combined by summing them (Section 3.4). This score definition remains algorithm-agnostic, allowing for direct comparison of anomaly detectors using different algorithms.

To further characterize model outputs, we introduced a visual representation of network anomalies (Section 3.5) that highlights both spatial aspects (e.g., whether anomalies affect or originate from the same IP address) and temporal aspects (e.g., whether anomalies are recurrent). Finally, our meta learner, a CNN, analyzes these anomaly representations to identify attack patterns and raise alerts (Section 3.6).

The system architecture is organized as follows:

- Network flow aggregation: Traffic data is collected and grouped by source and destination.
- Ensemble Anomaly scoring: Each aggregate is assessed by multiple UL models that compute an anomaly score.
- Anomaly score visualization: For each time interval, the anomaly scores produced by each model are visualized as monochromatic segments,

where the color intensity reflects the anomaly level. These segments, each representing a single model, are then vertically stacked to form a color-encoded segment that captures the ensemble analysis for that interval. Finally, these color-encoded segments are placed side by side to illustrate the temporal evolution of anomalies.

- Attack pattern recognition: Visual representations of anomalies are analyzed by a CNN. The CNN detects patterns in these images to identify attack signatures and trigger alerts for identified attacks.

3.2. Network Flow Aggregation

Network traffic data, captured by monitoring devices such as network probes, encompasses large volumes of raw information. These probes can capture traffic and extract detailed characteristics from network flows, which are typically structured with hundreds of features. This data richness, while comprehensive, can hinder analysis by overwhelming machine learning models with high dimensionality and by burdening analysts who lack the time to review every feature in the event of an alert. To address these challenges, we adopt and extend the aggregation strategy proposed in UNADA [8, 9], focusing on interpretable statistical features grouped by source and destination IPs.

To provide comprehensive insights into network traffic, we chose to aggregate flows by both source and destination IP addresses, focusing only on internal IPs within the company's network. This focus aligns with operational security practices where internal devices are closely monitored to detect signs of compromise or suspicious behavior. By limiting aggregation to internal IPs, we reduce the data volume, allowing for a more targeted analysis of potentially compromised assets within the network without overwhelming the system with external traffic information.

Aggregating flows by both source and destination addresses provides two complementary perspectives on network activity. For example, a distributed denial-of-service (DDoS) attack might appear as multiple sources targeting a single destination, indicating coordinated efforts to overwhelm a specific asset. Conversely, a network scan often involves a single source probing multiple destinations, likely in search of vulnerabilities. This decomposition not only aids in attack type identification but also highlights specific machines involved, helping security analysts better understand and respond to threats.

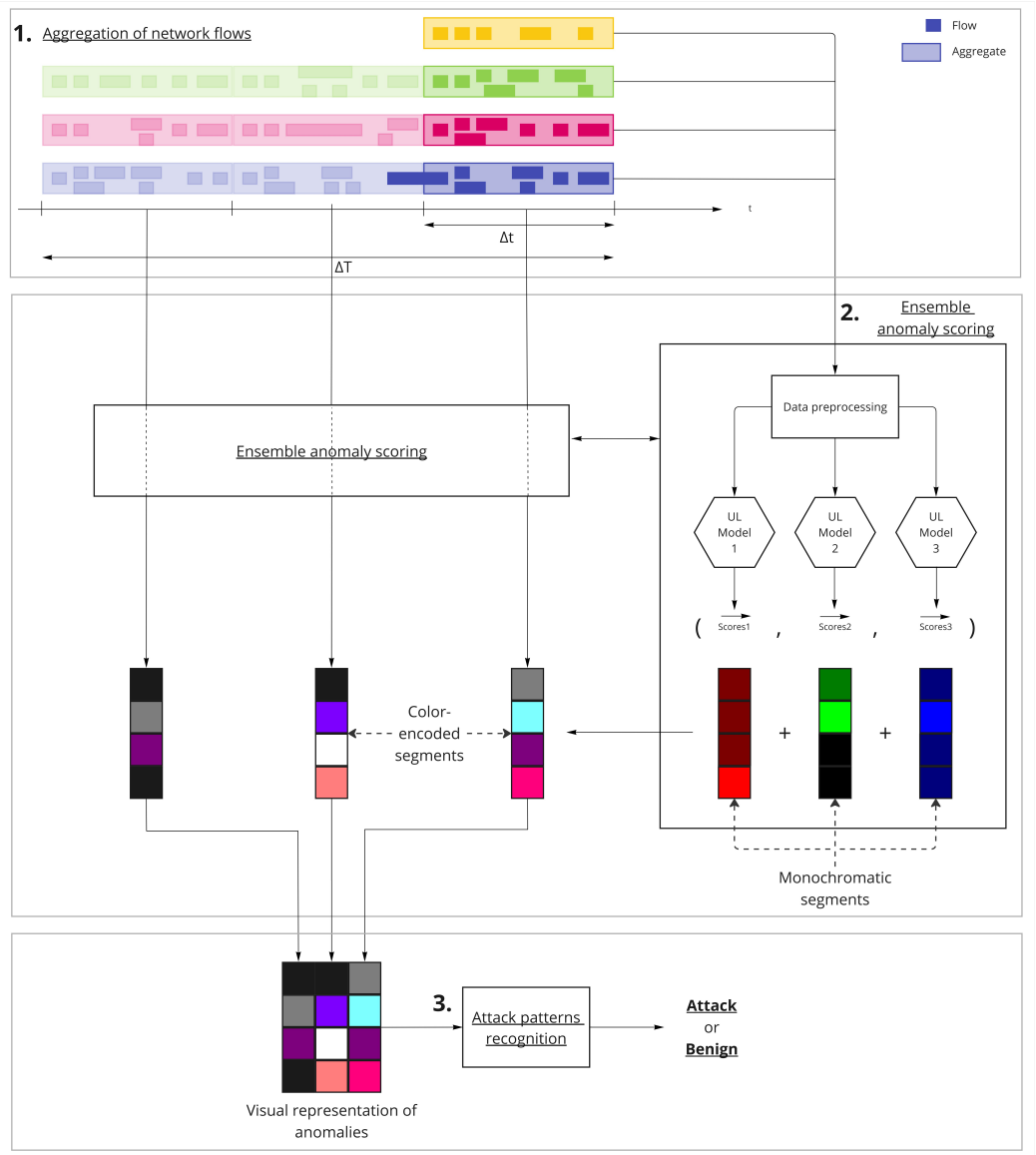

Figure 1: System overview. The system analyzes a traffic capture over a period Δ_T. It first aggregates network flows within time intervals Δ_t and computes their features. Next, it applies a set of $N = 3$ unsupervised base learners to assign anomaly scores to each aggregate. A visual representation of detected anomalies is then generated, providing a clear view of potential attack patterns in the capture. Finally, these representations are analyzed by an attack pattern recognition module, determining if the network is under attack during period Δ_T.

To manage variations in network traffic that can be mistaken for anomalies—such as changes due to remote work shifts or daytime versus nighttime traffic—we perform aggregation within fixed time intervals, Δ_t, and analyze abnormal behaviors independently within each interval. This strategy, inspired by UNADA [8, 9], avoids defining a static baseline, instead isolating unusual activities based on their characteristics within each segment.

We empirically determined the interval duration to balance capturing enough data without diluting potential attack signals. Short intervals may lack sufficient data for meaningful aggregation, while long intervals risk concealing malicious traffic within overall legitimate activity. After various tests, we set the interval duration, Δ_t, to 2 minutes, which offers optimal detection in our context. This duration could, however, require adjustment when applied to different datasets or environments. This fixed windowing strategy also allows the system to operate in a near-real-time manner. Although the analysis is performed only at the end of each 2-minute window, this short delay offers a reasonable trade-off between responsiveness and interpretability. Such a configuration is suitable for exploring practical detection scenarios without requiring real-time constraints.

Our system computes a set of features (Table 1) for each aggregated flow, specifically chosen for their potential to reveal attack patterns and ease of interpretation. These features include straightforward statistics such as packet counts and packet size sums, which provide insight into traffic volume and behavior. For instance, high packet counts of small sizes may indicate network scans or brute-force attempts, where an attacker probes defenses by sending frequent requests. Additionally, the number of forwarded and backward packets (n_fwd_pkts, n_bwd_pkts) can expose asymmetric communications, a characteristic of command and control (C&C) channels. An unusually high number of destination ports accessed by a single IP may signal a port scan, a tactic attackers use to identify open services on a network. By focusing on these key features, the system provides security analysts with immediately actionable insights.

Although our aggregation method is based on prior work such as UNADA [8, 9], it plays a central role in enabling our visual anomaly detection pipeline. By simplifying high-volume network data into structured, semantically meaningful aggregates, it supports a clear, interpretable, and efficient representation of network behavior. This aggregation strategy reduces complexity and noise, facilitating threat detection and offering security analysts an accessible and actionable view of network activity.

Table 1: Aggregates features

Feature	Aggregation key	Description
n_dst_ip	IPsrc	Number of destination IP addresses
n_src_ip	IPdst	Number of source IP addresses
n_dst_ports	IPsrc & IPdst	Number of destination ports
n_src_ports	IPsrc & IPdst	Number of source ports
n_fwd_pkts	IPsrc & IPdst	Number of forward packets
n_bwd_pkts	IPsrc & IPdst	Number of backward packets
sum_flx_dur	IPsrc & IPdst	Sum of flows duration
tot_flx	IPsrc & IPdst	Number of flows
sum_pkts_size	IPsrc & IPdst	Sum of packets size
std_pkt_size	IPsrc & IPdst	Standard deviation of packets size

3.3. Unsupervised Anomaly Scoring

This section outlines the method our system employs to assign an anomaly score to each aggregate using a single unsupervised anomaly detection algorithm. This scoring step serves as a foundation for our ensemble approach (detailed in Section 3.4), which integrates multiple UL models to provide a comprehensive view of network anomalies.

Our NIDS adopts an ensemble approach, leveraging the combined insights of various unsupervised learning models. Each model offers a unique perspective on network data, enhancing the overall understanding of network traffic anomalies. However, for effective integration and comparison of these models' results, we need an algorithm-agnostic scoring method.

We employ a statistical scoring approach based on the analysis of feature subsets, focusing on feature pairs instead of all available features. This approach directly builds on the anomaly scoring method developed in UNADA [8, 9] and is particularly well-suited for our objectives.

By analyzing reduced-dimensional spaces, this method not only improves anomaly detection performance by reducing data complexity but also enhances explainability. High-dimensional analyses often lack interpretability for security analysts. Limiting the dimensionality of analysis yields more understandable results, allowing analysts to clearly identify signals of anomalous behavior in network data.

The anomaly scoring process in our system consists of the following steps:

1. Feature pair selection: From each aggregate, the system identifies all $k = 2$ feature pairs among the $n = 9$ aggregate features (Table 1). The system processes two types of aggregates: aggregates by source and aggregates by destination, each containing 9 featuress. Among these features, 8 are common between the two perspectives, while 1 is specific to each perspective. Specifically, the aggregates by source include the number of destination IPs, whereas the aggregates by destination include the number of source IPs. Each feature pair represents a dimension of network traffic to analyze.
2. Algorithm application: The same anomaly detection algorithm, such as Isolation Forest [20], Local Outlier Factor [7], or DBSCAN [10], is applied independently to each feature pair.
3. Score aggregation: For each aggregate, we compute a raw anomaly score as the sum of binary values across all feature pairs, where $s_i = 1$ if the i-th feature pair is considered anomalous, and $s_i = 0$ otherwise:

$$s = \sum_{i=1}^{C} s_i$$

   where $C = \binom{n}{k} = 36$ is the total number of feature pairs. This raw score reflects how many feature pairs label the aggregate as anomalous.
4. Score Normalization: To obtain a standardized anomaly score, the raw score is normalized to the range $[0, 1]$:

$$s_{\text{norm}} = \frac{s}{C}$$

   This normalized score reflects the proportion of feature pairs in which an aggregate is considered anomalous: a normalized score s_{norm} close to 1 indicates a highly anomalous aggregate, while a score near 0 indicates normal behavior.

This method produces standardized, algorithm-independent anomaly scores, facilitating consistent anomaly evaluation across aggregates. It also enables seamless integration of scores into our ensemble framework, allowing them to be combined and compared to improve overall anomaly detection.

3.4. Ensemble Anomaly Scoring

This section details our strategy for selecting and combining various unsupervised anomaly detection algorithms to maximize the effectiveness of our

NIDS. By leveraging the complementary strengths of different algorithms, our ensemble approach aims to robustly detect a wide range of anomalous behaviors and potential new attacks.

To enhance our system's ability to detect new attacks, we rely on unsupervised anomaly detectors as base learners. We evaluated multiple unsupervised anomaly detection algorithms implemented in scikit-learn [26] and PyOD [35] and applied them to aggregates derived from the CSE-CIC-IDS2018 dataset [29]. This evaluation (Section 4.4) showed that, in certain cases, each algorithm provides distinct and complementary results when applied to the same data aggregates, highlighting the potential benefits of an ensemble approach..

The algorithms we selected for this study are diverse to maximize their complementary capabilities. The selected algorithms are:

- Isolation Forest (IF) [20], a tree-based algorithm,
- Local Outlier Factor (LOF) [7], which leverages density-based measures,
- DBSCAN [10], a proximity-based clustering algorithm,
- One-Class SVM (OCSVM) [28], a hyperplane-based algorithm,
- Unsupervised KNN [4], a distance-based method,
- COPOD [19], a probabilistic algorithm.

To construct an ensemble that balances high detection rates with minimal false positives (i.e., legitimate traffic mistakenly flagged as malicious), we selected a subset of base learners using the same input data but different anomaly detection algorithms. The "stacked model" combines these base learners by performing a boolean sum of their results, which aggregates alerts across models.

We evaluated all combinations of three algorithms, based on true positive rate (TPR) and false positive rate (FPR). This selection process on the CSE-CIC-IDS2018 dataset (see Section 4.5) empirically favored the combinations (LOF, OCSVM, COPOD) for aggregates by source and (LOF, KNN, COPOD) for aggregates by destination. The choice of these combinations can be explained by the diversity of the detection mechanisms employed by the algorithms, which allows for broader coverage of potential attacks.

Given the conservative nature of the stacking approach, which tends to generate numerous false alarms, the anomaly scores generated by this "stacked model" are further analyzed by the attack pattern recognition module to refine detection accuracy (see Sections 3.5 and 3.6).

3.5. Anomaly Visualization and Interpretation

After anomaly scores are assigned to each aggregate, we visualize these scores to create a clear, interpretable representation of the analyzed data. This visualization allows security analysts to rapidly identify patterns and potential threats.

For each time interval analyzed by a model, we generate a monochrome segment. In this segment, each line represents a specific IP address (source or destination) within the network, and the line's color intensity corresponds to the anomaly score assigned to that IP by the model for the given time interval. Darker shades, such as black, indicate normal activity with low anomaly scores, while brighter colors represent increasingly abnormal activity.

The monochrome segments from each model are then layered together to create a single, color-encoded segment that reflects the combined analysis from multiple models for the time interval. Each model's unique color contribution in the segment allows analysts to distinguish which model detected specific anomalies.

Once individual intervals are encoded, the colored segments are placed sequentially to show anomaly evolution over a longer period. This arrangement enables the observation of temporal patterns and the persistence of anomalies across multiple intervals.

In the visual representation of anomalies (Figures 2, 3 and 4), the X-axis shows time progression, while the Y-axis represents individual IPs within the network. Brightly colored pixels signal high anomaly scores, with their distribution revealing distinct attack patterns. Thus, a concentration of anomalies along the Y-axis for a single time interval often suggests multiple IPs targeted simultaneously or coordinated activity within the network. In contrast, a concentration of anomalies for a single IP address, often represented as a horizonal line of bright colors in consecutive segments, can indicate a repeated, persistent targeting of that specific IP address over time.

3.6. CNN for Attack Pattern Recognition

In the previous section, we identified attack patterns within our visual representations and provided illustrative examples. Building on this, the

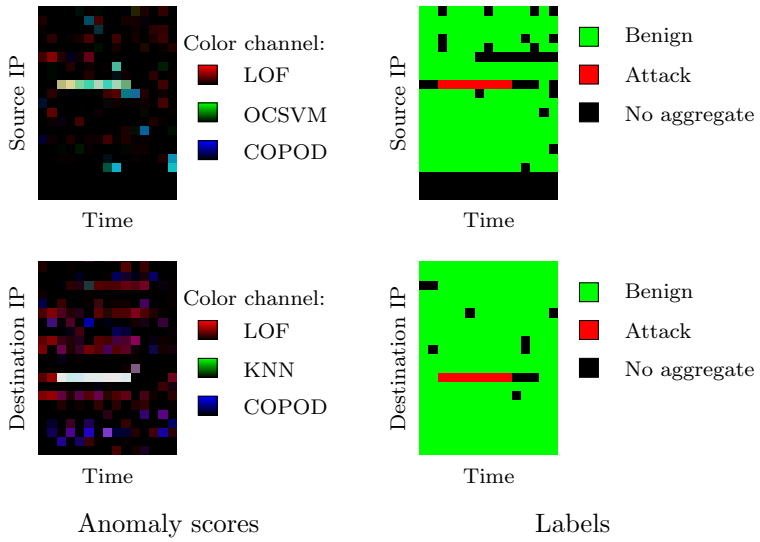

Figure 2: DoS attack. In the label representations, a horizontal red line indicates the attack flows sent and received closely by the victim. In the destination IP representation, the UL models detect the entire attack line as highly abnormal, with a few benign, slightly anomalous aggregates. In the source IP representation, the attack line is detected as moderately abnormal.

current section details our method for automatically detecting these patterns using deep learning models, specifically CNNs. CNNs are well-suited for discerning patterns in visual representations generally, thus, we will explore their effectiveness for detecting cyber-attack patterns in our specific case.

To prepare the input data for the CNN, we use visual representations that capture network activity over a time period, Δ_T. As observed in the previous section, representations of source and destination aggregates provide complementary perspectives for attack detection. Thus, our CNN is designed to process a pair of images—one for aggregates by source and one for aggregates by destination—over a single period, Δ_T.

Each image is labeled as "attack" if it contains any malicious traffic during the capture period, and "benign" otherwise. The CNN is then trained to recognize and classify traffic periods based on the presence or absence of malicious activity. This approach allows the CNN to identify complex attack patterns using these structured visual data.

Regarding our CNN architecture, we employ a classical setup with two sets of convolutional layers that analyze the source and destination perspec-

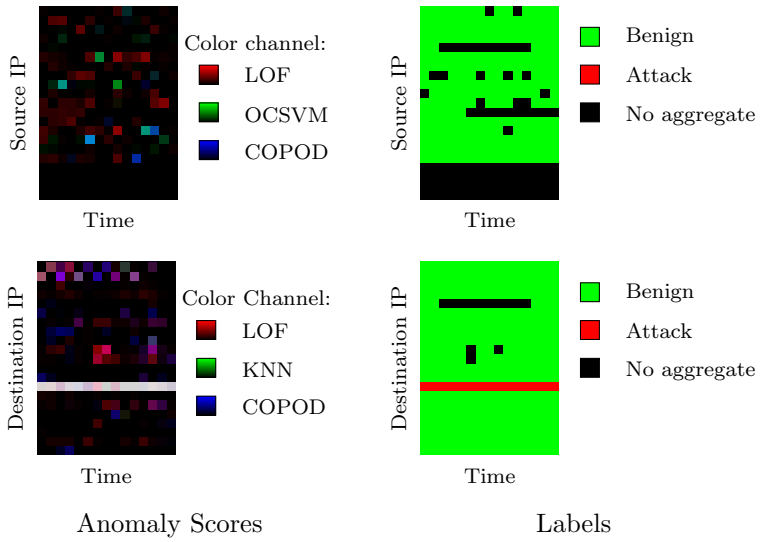

Figure 3: Brute-force attack. In the label representations, the victim receives attack flows but does not emit them. The attack is only visible in the destination IP representation, where the UL models detect the entire attack line as highly anomalous, with a few benign, slightly anomalous aggregates.

tives separately. This configuration allows each model to detect and specialize in patterns specific to either source or destination. The outputs from these models are then combined through a single concatenation layer, enabling the detection of attacks by considering both perspectives simultaneously.

Our system employs unsupervised detectors to identify new attacks, and enhances this detection with a supervised CNN that acts as a meta-learner. This CNN does not directly analyze raw network flows, which are complex and context-dependent. Instead, it learns from anomaly scores generated by the unsupervised detectors. These scores, representing higher-level metadata, are less sensitive to specific variations in network data and enable the CNN to detect simple, general patterns indicative of abnormal behaviors, such as continuous or isolated anomalies in the anomaly score image. Through this meta-learning approach, our system is equipped not only to recognize known attack types but also to adapt to emerging threats that exhibit similar, though novel, anomalous characteristics. By focusing on generalizable patterns rather than specific flow signatures, our model better adapts and generalizes to new threats, maximizing its capability to detect emerging attacks that might otherwise evade traditional detection methods.

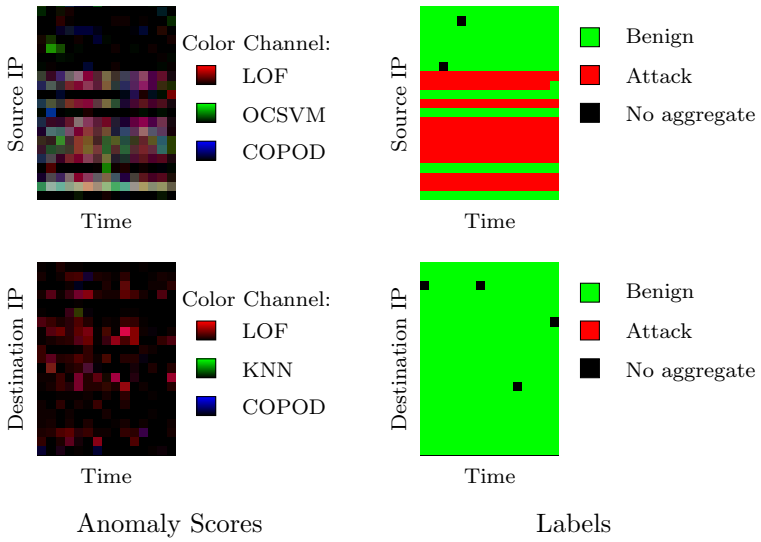

Figure 4: Botnet attack: This visual representation highlights a distinct pattern characteristic of a botnet attack. The lower segment has a dark red hue, suggesting a low-level anomaly detected by one of the models. Unlike the clear and intense signals typical of brute-force or DoS attacks, botnet activity appears as a more diffused and extended pattern, reflecting its stealthy nature. The anomalies are not sharply defined but form an unusual and elongated block across the timeline, hinting at the botnet's persistent and covert behavior. This subtle yet consistent anomaly pattern warrants further investigation, as it may indicate a long-term compromise aiming to evade detection while performing illicit activities like data exfiltration.

In practice, once deployed, the visual representations of anomalies are presented as a sequence to analysts, simulating a video stream. The CNN assesses each image representing a period Δ_T. If an alarm is triggered for Δ_{T1} after a prior period Δ_{T0} without alarms, this indicates recent anomalies appearing in the latest columns of the image, suggesting an emerging threat. Analysts can then examine the relevant IP addresses and review the specific feature pairs that triggered the alarm to analyze the nature and context of the attack in detail. This method aims to facilitate rapid and informed intervention.

3.7. A System Designed for Explainability

In ML-based NIDS, explainability is critical not only for analysts to understand and validate generated alerts but also for system designers to identify potential errors and biases in data. This section outlines how our system is built with explainability at its core, emphasizing the methods and architectural aspects that make the detection process transparent and accessible to end users while enhancing their trust in automated decisions.

3.7.1. Explainability-Oriented Architecture

Our system's architecture emphasizes a structured, modular approach that segments anomaly analysis into distinct steps, each mirroring the analytical processes a human expert might undertake with network data. This modular segmentation begins with data collection and aggregation, followed by anomaly scoring using unsupervised anomaly detectors. Each detector processes data independently, allowing for granular analysis and precise identification of abnormal behaviors. Finally, anomalies detected by the various detectors are presented on a heatmap, which a CNN analyzes to simulate the role of a security analyst assessing the network's health.

In this system, each architectural component contributes intermediate results—ranging from anomaly detector outputs to visualizations and CNN interpretations—that are compiled into a cohesive report. This report provides a complete trace of the detection process, helping security analysts understand the rationale behind each alert and decide whether to confirm or dismiss it. By structuring detection results in this way, the system ensures transparency and enables analysts to trace back each alert to specific indicators, supporting more confident, informed decisions.

For data engineers, the system's modular structure also enhances explainability by isolating each processing stage—from data aggregation to visual-

ization and classification. This organization makes it easier to analyze or adjust a specific module without affecting the rest of the system, facilitating iterative improvements based on feedback or evolving security requirements. It also enables fine-grained performance evaluations of each component, supporting robust system maintenance and evolution.

3.7.2. Visualization and Dimensionality Reduction

To enhance explainability, our system prioritizes visualization and dimensionality reduction principles that help analysts in interpreting anomalies.

The system uses a set of clearly defined features (nine per aggregate type) to describe network traffic aggregates, making anomaly detection accessible and relevant in operational contexts. These selected features allow analysts to quickly identify critical aspects of data, ensuring efficient, targeted investigations without information overload, thus enabling a faster response to incidents.

The chosen features are carefully selected for their relevance and interpretability for security analysts. Unlike approaches that maximize machine learning model performance using abstract features, our focus is on features that have direct meaning within network security analysis. This ensures that the system's results are not only accurate but also comprehensible and actionable for professionals.

Reducing anomaly detection to two-dimensional spaces is another aspect that enhances the system's explainability. High-dimensional anomalies are often impossible for humans to interpret. By projecting data into a 2D space, we make anomaly patterns not only visible but also simpler to analyze. This transformation helps analysts in visualizing and understanding abnormal behavior without requiring advanced statistical expertise in complex data processing.

Our use of images to represent network anomalies, displaying anomaly scores from each detector, offers a powerful tool for quickly identifying affected machines and event times while revealing attack patterns. These visualizations make abnormal behaviors visible and easily interpretable, allowing analysts to quickly grasp network anomalies, detect trends, and pinpoint potential errors or biases in data models.

3.7.3. Automated Interpretation with CNNs

While CNNs are often perceived as opaque, their integration in our system does not conflict with our explainability goals. Rather than applying

the CNN directly to raw network data, we use it to analyze compact visual representations specifically designed with explainability in mind. These visual representations map anomalies in a way that makes patterns visible and interpretable to humans, thus supporting CNN interpretation.

This approach ensures that even though the CNN may operate as a "black box", its inputs and outputs remain in a format easily understood by analysts. For clearly identifiable attacks, such as a sustained DDoS, the CNN provides quick confirmation without human intervention, allowing analysts to focus on more complex cases. For subtler patterns or emerging attack types, human verification remains essential, maintaining a balance between efficient automation and expert oversight.

4. Experimental Validation

This section presents the experimental evaluation of our system, focusing on both its detection capabilities and the interpretability of its outputs. The objective is to assess how well the architecture supports anomaly detection in realistic network settings, and to understand the behavior of each component under real-world-like conditions.

All results are based on a single execution of the full pipeline. This was a deliberate choice, motivated by our emphasis on analyzing the system's qualitative behavior and its explainability features, rather than on exhaustive statistical performance metrics. Conducting a single run allowed for detailed inspection of anomaly patterns, model interactions, and visual outputs.

4.1. Dataset Selection

Choosing the right dataset is critical for accurately assessing the performance of attack detection systems like ours. For evaluating our system, we selected the CSE-CIC-IDS2018 dataset [29], chosen for its comprehensive and representation of modern threats, high-quality labeling, and inclusion of both benign and malicious traffic. This dataset meets our criteria for evaluating unsupervised anomaly detection techniques in a realistic network environment.

The CSE-CIC-IDS2018 dataset [29] provides raw pcap files, capturing ten workdays of simulated enterprise network activity in a realistic setting with 420 machines and 30 servers. The accompanying detailed documentation[1]

[1]https://www.unb.ca/cic/datasets/ids-2018.html

outlines attack timelines and attacker-victim IP mappings, which allow the identification of attacks within the pcap files.

The dataset covers a diverse range of attacks frequently encountered in enterprise networks, including brute-force attacks, DoS and DDoS aimed at overwhelming network resources, SQL injections, and web attacks exploiting application vulnerabilities.

While CSE-CIC-IDS2018 offers a valuable foundation for evaluating detection techniques, it does have some limitations. The dataset captures traffic only during weekday work hours, limiting analysis of off-peak behaviors, and lacks specific contextual details about the network infrastructure, such as machine roles or departmental affiliations.

Despite its relative age and the evolution of recent threat models, the CSE-CIC-IDS2018 [29] dataset remains a standard benchmark in anomaly detection research. It features continuous traffic and labeled attack scenarios that are sufficient for evaluating our method's visual interpretability, and also the effectiveness of the ensemble-based unsupervised detection strategy we propose.

4.2. Network Flow Aggregation Process

This section outlines the network flow aggregation approach (Section 3.2) applied to the CSE-CIC-IDS2018 dataset [29], a key step in preparing data for evaluating our unsupervised attack detection system.

To avoid potential labeling errors from CICFlowMeter [17], on the one hand, and to include IP addresses in our analysis, on the other hand, we did not directly use the labeled network flows from CSE-CIC-IDS2018 [29]. Instead, we used the Custocy probe [1] to extract network flows and relevant features for our analysis.

Our data extraction and preparation process began with a thorough review of the raw pcap files and the accompanying documentation for the CSE-CIC-IDS2018 dataset [29], which includes attack timings and attacker and victim IP addresses. Leveraging this information, we segmented the pcap files to create separate files for each type of event. We conducted a detailed inspection of each segment to ensure accurate labeling and prevent potential labeling errors that could arise if data were misinterpreted or misaligned with documented events. Each pcap segment was clearly marked as associated with a specific attack or normal traffic, based on the documented details.

Once these segments were identified, we passed all raw data through the Custocy probe [1], based on nProbe[2], to extract flow features. The Custocy probe [1] allowed us to capture detailed flow information, such as session durations, data volumes, protocol types, and, importantly, IP addresses that are absent from the initial features provided with the dataset.

After feature extraction, flows were aggregated separately based on source and destination IP addresses, resulting in two distinct sets of aggregates. To simplify analysis, especially to reduce dimensions in image processing, we focused exclusively on internal enterprise IP addresses. For aggregates by source, only flows originating from internal IPs were retained, while for aggregates by destination, only flows directed toward internal IPs were considered. This decision limited the data volume to be processed and targeted interactions most relevant to internal security, while also making it more practical for analysts to interpret reduced-dimension images. For each aggregate, we calculated statistical features from individual flows to offer a detailed analysis of traffic behavior within each aggregate.

For aggregate labeling, we used a straightforward, conservative approach: an aggregate was labeled as "benign" if all its flows were benign, and as "attack" if it contained at least one flow identified as part of an attack. This labeling strategy ensures that any aggregate containing even a single malicious flow is thoroughly analyzed, enhancing the likelihood of detecting malicious activities and minimizing the risk of overlooking potential attacks.

Table 2 describes the distribution of aggregates for each traffic category. We observe that the number of aggregates per source exceeds those per destination, indicating a higher number of internal machines initiating flows within this traffic capture. Additionally, certain attacks, such as web and bot attacks, are only visible either by aggregation by source or destination, but not by both. This underlines the need to consider both source and destination IP addresses in security analysis to capture the full spectrum of malicious activities.

For instance, bot attacks involve regular data exfiltration from infected machines, explaining the absence of destination aggregates in this category. Attackers use bots to automate data exfiltration, initiating flows from infected machines to external destinations under their control. In contrast, web attacks, represented in our dataset by attacker-initiated interactions,

[2]https://www.ntop.org/products/netflow/nprobe/

Table 2: Distribution of Aggregates by Source and Destination IP Address for Each Traffic Category

	Proportion (%)	
Traffic Category	**Source IP**	**Destination IP**
Brute-force Attack	0.004455	0.009547
Denial of Service	0.001412	0.004231
Web Attack	0	0.013089
Infiltration	0.003477	0.007480
Bot	0.162978	0
Distributed Denial of Service	0.006302	0.006102
Benign	99.821376	99.956894

are only observed in aggregates by destination. During the simulation of these attacks, flows were initiated by an automated script, allowing the attacker to systematically launch attacks such as SQL injection and cross-site scripting (XSS).

Lastly, benign traffic dominance, exceeding 99% of aggregates here, is essential for enabling unsupervised anomaly detection techniques to identify attacks. The dataset is therefore highly imbalanced, with benign traffic representing the overwhelming majority of samples. Rather than correcting for this imbalance, we preserve it deliberately to reflect realistic deployment scenarios, where attacks are rare and must be detected among mostly legitimate traffic.

4.3. Unsupervised Anomaly Scoring

This section evaluates the application of multiple base learners for anomaly detection on aggregates (Section 3.3). Each model produces an anomaly score per aggregate, indicating the likelihood of anomalous behavior. This unsupervised approach forms the foundation of our ensemble detection method.

The base learners are a selection of unsupervised anomaly detection algorithms (Section 3.3), chosen to provide a wide methodological range that captures various aspects of network traffic anomalies.

The unsupervised models include algorithm-specific hyperparameters, such as the number of trees (n_estimators) in Isolation Forest and the number of neighbors (n_neighbors) in Local Outlier Factor. These parameters are generally optimized iteratively, testing various values to improve performance

metrics—a resource-intensive process, especially challenging due to the lack of precise data labeling in real-world traffic environments.

Given the practical difficulty of fine-grained labeling in real-time enterprise traffic, exhaustive hyperparameter tuning was not feasible. Enterprises typically lack access to adequately labeled network traces necessary for detailed parameter adjustments. Therefore, we retained default parameter values, making only minor adjustments to enhance model generalization and robustness across varied traffic patterns:

- Contamination parameter: This parameter specifies the proportion of data expected to be anomalies. The contamination rate was set empirically at 0.05 (5%) based on both visual and quantitative analysis (Section 4.5). Although this rate may seem high compared to the observed attack proportions in the dataset—0.18% for source IP aggregates and 0.05% for destination IP aggregates (Table 2)—the higher threshold was chosen to ensure comprehensive coverage of anomalous behaviors, including those that are subtle yet potentially harmful. This approach is designed to capture a significant number of attacks, recognizing that it might increase false positives. However, the management of these false positives is intended to be addressed by a subsequent component of our system (Section 4.6), which will refine detection accuracy.

- Minimum number of data points: Beyond the contamination parameter, a few other hyperparameters were adjusted:

  - n_neighbors for KNN: Set to 3 to accommodate smaller aggregate samples within certain time intervals.
  - min_samples for DBSCAN: Set to 4 to allow for clustering even with low aggregate counts per interval.

  These values were chosen to ensure anomaly detection viability even in scenarios with limited data aggregates. For instance, at least four aggregates per time interval were needed for detection to remain meaningful and actionable, helping maintain detection efficacy under sparse data conditions.

To assess base learners performance, two key indicators were measured:

- TPR: The percentage of correctly identified attacks.

Table 3: Attack Detection (TPR) and False Alarm Rates (FPR) for base learners

	Source IP		**Destination IP**	
Base learner	**TPR**	**FPR**	**TPR**	**FPR**
IsolationForest	0.6630	0.1462	0.8630	0.1636
LocalOutlierFactor	0.8923	0.3754	0.9041	0.3366
DBSCAN	0.2007	0.1032	0.7808	0.0696
OneClassSVM	0.8394	0.4634	0.8676	0.2375
KNN	0.2579	0.1568	0.8904	0.1862
COPOD	0.7652	0.1467	0.8744	0.1795

- FPR: The percentage of false alarms generated by the model.

Table 3 summarizes the TPR and FPR for each base learner when applied to aggregates by source and destination. Results show the limitations of each model individually: while LOF achieved high detection rates, it also yielded a substantial number of false positives, potentially overwhelming analysts with non-critical alerts. Similarly, for aggregates by destination, LOF continued to perform well on detection but also generated high false positives, reducing its practical utility.

Individually, none of the models provided satisfactory results. The best-performing model, LOF, still missed nearly 10% of attacks in aggregates by destination, a critical gap for effective security monitoring. Moreover, its high detection rate was offset by frequent false alarms, complicating discernment for security analysts between actual threats and benign anomalies.

This analysis highlights the need for advanced techniques that can blend the strengths of multiple models. In the following section, we explore consensus across models to determine whether complementary detections can improve overall system performance.

4.4. Agreement and Disagreement Analysis

This section explores how different anomaly detection models either agree or disagree when classifying aggregates as benign or attack-related. Understanding these areas of agreement and disagreement helps us enhance our ensemble approach.

We define "agreement zones" as cases where all models classify an aggregate consistently as either benign or anomalous. However, unanimity does

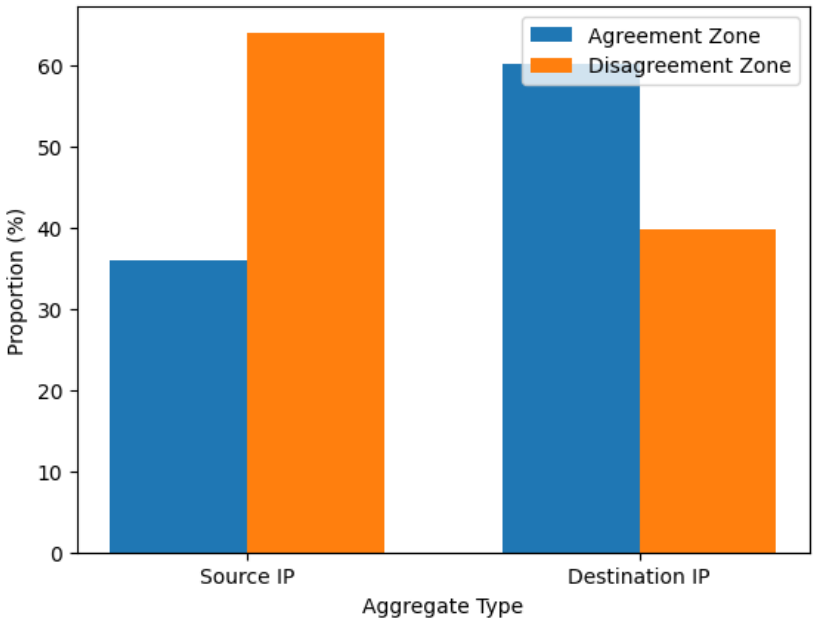

Figure 5: Distribution of aggregates within agreement and disagreement zones

not guarantee accuracy, as all models may still misclassify certain aggregates, posing reliability risks for the detection system.

"Disagreement zones", on the other hand, arise when at least one model disagrees with the others. These zones are essential for understanding the differing sensitivities of models, particularly when handling complex scenarios. By analyzing these areas, we aim to leverage the unique strengths of each model, especially in cases where models diverge.

As shown in Figure 5, about 36% of source IP aggregates and 60% of destination IP aggregates fall within agreement zones. This distribution underscores the importance of disagreement zones, as they may contain subtle anomalies or edge cases not uniformly detected by all models. By addressing these divergences within a stacked framework, we can improve overall detection performance, reducing false positives and false negatives.

For aggregates by source in the disagreement zone (Figure 6), a higher proportion of samples are detected by three or four models, indicating a pattern of partial consistency across models. However, several cases are identified by only one or two models, reinforcing the need for a stacked approach that can handle these variances.

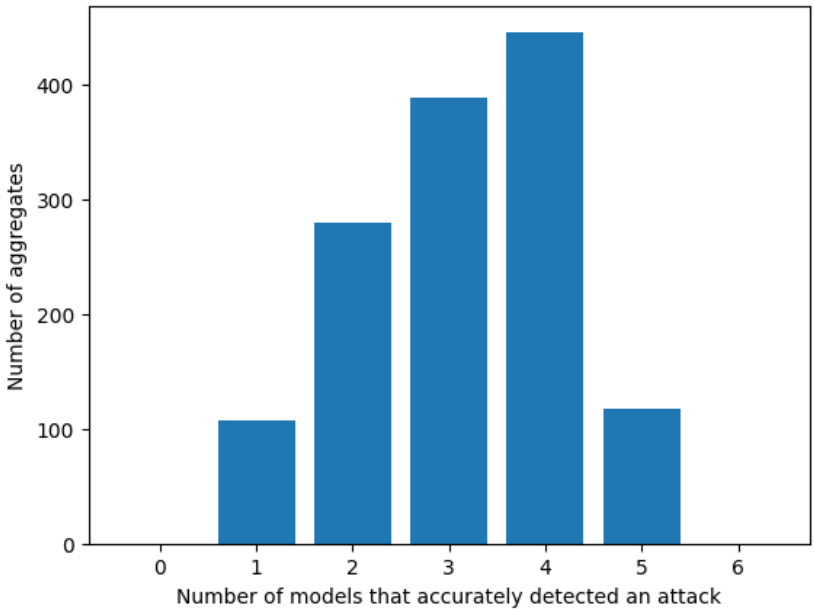

Figure 6: Distribution of attack aggregates by source located in the disagreement zone, based on the number of base models that correctly detected them. The x-axis indicates how many models flagged each aggregate as anomalous, and the y-axis shows how many aggregates fall into each category. For example, the bar at $x = 3$ indicates how many attack aggregates were correctly detected by exactly three models.

For aggregates by destination (Figure 7), many attacks are detected by only one model, highlighting the limitations of majority voting. In scenarios where a single model out of five detects an attack—what we refer to as "minority reports"—relying solely on a majority decision could lead to missed detections.

In minority report cases (Figure 8), where only one model detects an attack, LOF performs best, while models like COPOD and KNN also contribute value. This variability suggests that even models with lower global performance can provide essential insights in specific scenarios.

This analysis underscores the limitations of majority voting and weighted voting methods in ensemble models. While agreement zones often reflect consensus-driven errors such as false negatives and over-detection, disagreement zones highlight the critical role of minority reports in identifying subtle anomalies that these voting methods might overlook. We implement a stacked approach that integrates these insights and significantly improves the

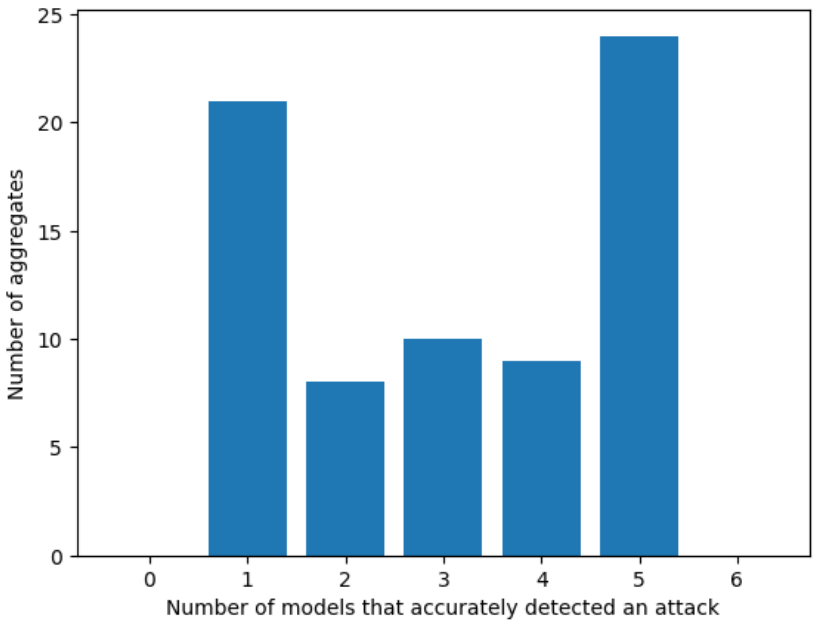

Figure 7: Distribution of attack aggregates by destination located in the disagreement zone, based on the number of base models that correctly detected them. The x-axis indicates how many models flagged each aggregate as anomalous, and the y-axis shows how many aggregates fall into each category. For example, the bar at $x = 3$ indicates how many attack aggregates were correctly detected by exactly three models.

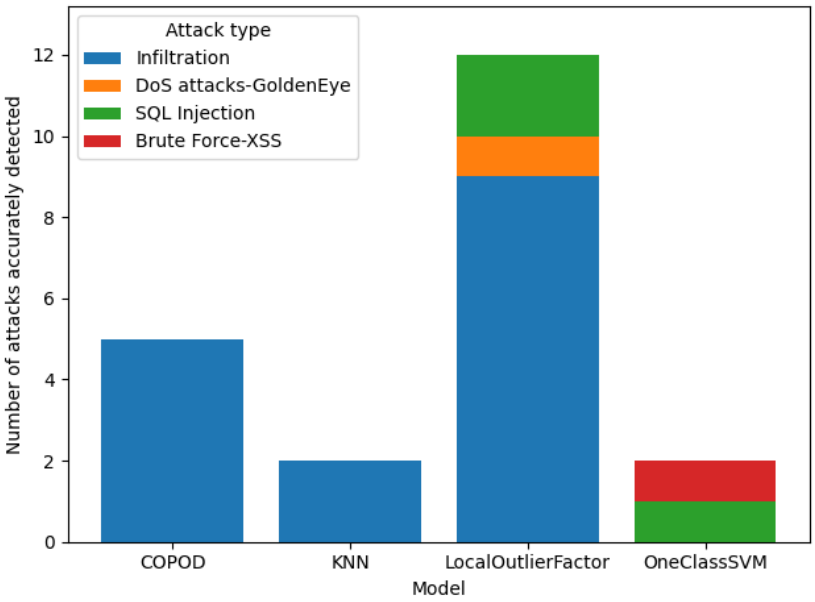

Figure 8: Number of attacks correctly detected by a single model for destination aggregates

detection accuracy and reliability of the system.

4.5. Stacked Model Construction

This section details the construction of the stacked model (Section 3.4), a critical step in our approach to unsupervised network anomaly detection. We select a set of complementary algorithms, each offering unique perspectives on anomaly detection.

Each base learner in our ensemble is associated with a specific type of aggregate (either source or destination) and applies the same anomaly detection algorithm to all feature pairings within that aggregate. Since each type of aggregate is described by 9 features, there are $\binom{9}{2} = 36$ possible feature pairs. The base learner analyzes each of these pairs independently, producing a binary result (0 or 1) to indicate whether the corresponding subspace is anomalous or benign. The binary results are then summed, resulting in an anomaly score for the base learner that ranges from 0 (no anomalies detected) to 36 (anomalies detected across all feature pairs). The stacked model combines multiple base learners, each employing a distinct anomaly detection algorithm, to capture complementary aspects of anomalous behavior (see Section 3.3).

To create the stacked model, we adopted a conservative approach: an aggregate is classified as an anomaly if at least one base learner detects it as such. In practice, this means that if any single model assigns a score above 0, the aggregate is flagged as anomalous. This approach leverages each model's strengths, maximizing the chances of identifying attacks even if one model fails to do so.

4.5.1. Empirical Tuning of the Contamination Hyperparameter

The contamination parameter, critical to each base learner, directly affects the threshold for classifying an observation as an anomaly. Given the low attack proportion in our dataset, we tuned this parameter empirically to balance detection accuracy with the reduction of false alarms. Without tuning, excessive false alarms could overwhelm visualization tools, complicating analysis.

We prioritized attack detection over reducing false alarms, considering that missed attacks pose a higher risk. Through iterative testing, we adjusted the contamination parameter to find the optimal balance between high detection and manageable alert volumes. Tests (Tables 4 and 5) indicated that a contamination level of 0.05 captured a similar number of attacks to 0.06 but with significantly fewer false alarms, particularly for source aggregates. Thus, 0.05 was chosen to maximize precision while maintaining system usability.

4.5.2. Optimal Number of Base Learners

To build our stacked model, we explored all possible base learners combinations, evaluating their detection performance.

During testing, configurations with four base learners showed a slight improvement in TPR compared to those with three models (Tables 6 and 7). However, adding a fourth model for each perspective (source and destination aggregates) introduces two additional matrices to the analysis, significantly increasing the complexity of the CNN's task. Each base learner is represented by a matrix, and adding these matrices demands more processing without delivering substantial gains in detection performance. While the system is designed to work with any number of models, using three models has the added advantage of aligning naturally with RGB visualization, where each model corresponds intuitively to one of the red, green, or blue channels. This visualization simplifies interpretation but is not a constraint of the system, which remains flexible to accommodate more models if necessary.

Table 4: Best combinations of base learners with different contaminations for aggregates by source. The combinations are ranked according to true positive rates (TPR) and false positive rates (FPR).

Combination of base learners	**TPR**	**FPR**
$contamination = 0.04$		
(IF, LOF, OCSVM, COPOD)	0.9714	0.6433
(IF, LOF, DBSCAN, OCSVM, COPOD)	0.9714	0.6454
(IF, LOF, OCSVM, KNN, COPOD)	0.9714	0.6481
(IF, LOF, DBSCAN, OCSVM, KNN, COPOD)	0.9714	0.6490
$contamination = 0.05$		
(IF, LOF, OCSVM, COPOD)	0.9915	0.6796
(IF, LOF, DBSCAN, OCSVM, COPOD)	0.9915	0.6806
(IF, LOF, OCSVM, KNN, COPOD)	0.9915	0.6858
(IF, LOF, DBSCAN, OCSVM, KNN, COPOD)	0.9915	0.6861
$contamination = 0.06$		
(IF, LOF, OCSVM, COPOD)	0.9969	0.7144
(IF, LOF, DBSCAN, OCSVM, COPOD)	0.9970	0.7149
(LOF, OCSVM, KNN, COPOD)	0.9969	0.7207
(LOF, DBSCAN, OCSVM, KNN, COPOD)	0.9970	0.7209

Table 5: Best combinations of base learners with different contaminations for aggregates by destination. The combinations are ranked according to true positive rates (TPR) and false positive rates (FPR).

Subset of base learners	**TPR**	**FPR**
$contamination = 0.04$		
(LOF, OCSVM, COPOD)	0.9200	0.3888
(LOF, DBSCAN, OCSVM, COPOD)	0.9201	0.3899
(IF, LOF, OCSVM, COPOD)	0.9201	0.3902
(IF, LOF, DBSCAN, OCSVM, COPOD)	0.9201	0.3912
$contamination = 0.05$		
(LOF, OCSVM, KNN, COPOD)	0.9429	0.4495
(IF, LOF, OCSVM, KNN, COPOD)	0.9429	0.4500
(LOF, DBSCAN, OCSVM, KNN, COPOD)	0.9429	0.4502
(IF, LOF, DBSCAN, OCSVM, KNN, COPOD)	0.9430	0.4507
$contamination = 0.06$		
(LOF, OCSVM, KNN, COPOD)	0.9521	0.5016
(LOF, DBSCAN, OCSVM, KNN, COPOD)	0.9521	0.5020
(IF, LOF, OCSVM, KNN, COPOD)	0.95205	0.5021
(IF, LOF, DBSCAN, OCSVM, KNN, COPOD)	0.9521	0.5025

Table 6: Best combinations of 2, 3, and 4 base learners for aggregates by source. The combinations are ranked according to true positive rates (TPR) and false positive rates (FPR).

Subset of base learners	**TPR**	**FPR**
4 models		
(IF, LOF, OCSVM, COPOD)	0.9914	0.6796
(IF, LOF, DBSCAN, OCSVM)	0.9914	0.6806
(LOF, DBSCAN, OCSVM, COPOD)	0.9878	0.6778
(LOF, OCSVM, KNN, COPOD)	0.9878	0.6846
3 models		
(LOF, OCSVM, COPOD)	0.9878	0.6764
(IF, LOF, OCSVM)	0.9811	0.6704
(LOF, OCSVM, KNN)	0.9732	0.6762
(LOF, DBSCAN, OCSVM)	0.9726	0.6620
2 models		
(LOF, OCSVM)	0.9726	0.6587
(OCSVM, COPOD)	0.9611	0.4860
(LOF, COPOD)	0.9550	0.4598
(LOF, KNN)	0.9057	0.4656

Table 7: Best combinations of 2, 3, and 4 base learners for aggregates by destination. The combinations are ranked according to true positive rates (TPR) and false positive rates (FPR).

Subset of base learners	**TPR**	**FPR**
4 models		
(LOF, OCSVM, KNN, COPOD)	0.9429	0.4495
(LOF, DBSCAN, KNN, COPOD)	0.9385	0.4019
(IF, LOF, KNN, COPOD)	0.9384	0.4020
(LOF, DBSCAN, OCSVM, COPOD)	0.9384	0.4446
3 models		
(LOF, KNN, COPOD)	0.9384	0.3994
(LOF, OCSVM, COPOD)	0.9384	0.4437
(LOF, OCSVM, KNN)	0.9315	0.4312
(LOF, DBSCAN, COPOD)	0.9292	0.3917
2 models		
(LOF, COPOD)	0.9292	0.3882
(LOF, KNN)	0.9269	0.3727
(LOF, OCSVM)	0.9155	0.4164
(IF, LOF)	0.9132	0.3613

Two-model configurations were also tested (see Figure 9). Although they detected most attacks, visual representations of attack patterns were less distinguishable from false alarms than with three models (see Figure 2), making interpretation more challenging.

Based on these observations, we ultimately chose a three-model configuration, balancing detection accuracy, visualization clarity, and computational feasibility. The RGB-based visualizations effectively distinguished attack patterns, facilitating more intuitive and faster interpretation of results.

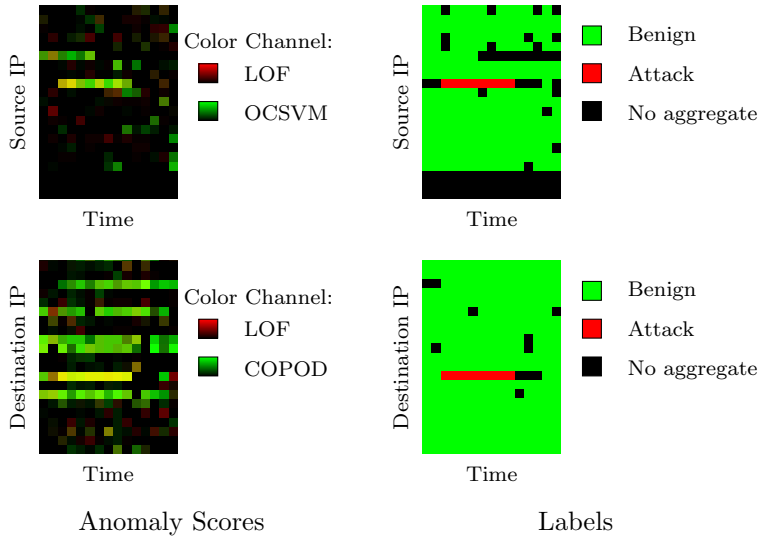

Figure 9: DoS attack detected using two base learners. In the label representations, a horizontal red line indicates the attack flows sent and received closely by the victim. In the source IP representation, two distinct bright lines stand out, indicating that the models have detected high levels of anomalies: one corresponds to the actual attack, while the other represents false alarms. In the destination IP representation, numerous bright lines appear for a single attack, making it challenging to distinguish the actual attack from false alarms. However, the attack line is still visibly brighter than the others, providing a subtle but discernible differentiation.

4.5.3. Selection of Base Learners

After establishing the contamination parameter and selecting three base learners, we identified the optimal model combinations.

We aimed to choose three models that maximized TPR while minimizing FPR, ensuring the stacked model effectively detected attacks while keeping false alarms manageable. Empirical testing showed that the combinations

Table 8: True Positive Rates (TPR) and False Positive Rates (FPR) for base learners and the stacked model

Source IP

Base learner	**TPR**	**FPR**
LOF	0.8923	0.3754
OCSVM	0.8394	0.4634
COPOD	0.7652	0.1467
Stacked Model	**0.9878**	**0.6763**

Destination IP

Base learner	**TPR**	**FPR**
LOF	0.9041	0.3366
KNN	0.8904	0.1862
COPOD	0.8744	0.1795
Stacked Model	**0.9384**	**0.3994**

of (LOF, OCSVM, COPOD) for source aggregates (see Table 6) and (LOF, KNN, COPOD) for destination aggregates (see Table 7) provided the best results. These models employ different anomaly detection methods, and in the case of destination aggregates, it has been observed (see Figure 8) that depending on the scenario, each model may be the only one to correctly detect certain anomalies. This explains why the stacked model that results from this combination covers a broader spectrum of anomalies than any single model could.

Table 8 presents TPR and FPR for each base learner and the resulting stacked model. Although LOF, the top-performing individual model, had a high TPR of 0.8923 for aggregates by source and 0.9041 for aggregates by destination, it did not catch all attacks detected by other models. The stacked model achieved the highest TPR, reaching 0.9878 for aggregates by source and 0.9384 for aggregates by destination, but with a slight increase in false alarms—exceeding the highest individual FPRs. This increase in false alarms will be addressed in subsequent analyses.

These results confirm the benefit of our ensemble approach: the stacked model consistently achieves higher true positive rates than any individual base learner, demonstrating improved anomaly coverage through model combination. While this higher sensitivity leads to an increase in false positives,

it is an expected trade-off at this stage, where the goal is to detect as many potential anomalies as possible. These preliminary detections are then refined by the CNN in the subsequent pattern recognition stage (Section 3.6).

In summary, the stacked model was constructed with careful tuning and selection, balancing detection accuracy, visual clarity, and computational efficiency. Designed to be adaptable, the model's structure meets both technical and operational requirements, ensuring smooth integration into Custocy's NDR solution [1] and reliable performance in real-world environments.

4.6. Attack Pattern Recognition with CNN

This section focuses on the final step of our approach: recognizing attack patterns based on visual representations generated from anomaly scores by the stacked model (Section 3.6). After assigning anomaly scores to aggregates and generating images that visualize these anomalies, the objective is now to train a module capable of detecting complex attack patterns within these representations.

4.6.1. Image Generation

In Section 4.5, each base learner assigned an anomaly score to each aggregate, which we use to generate monochrome segments per model and per time interval Δ_t. We then overlay the segments for each color corresponding to the same interval Δ_t and arrange them side by side to represent traffic over a longer period, Δ_T. This process yields images where, in this specific evaluation with three base learners, each color channel (R, G, B) corresponds to a different base learner.

Each generated image represents a 30-minute traffic capture with a resolution of 451 x 15 pixels. Each line in the image corresponds to an internal IP address, with columns representing 2-minute intervals within the 30-minute period. Higher color intensity corresponds to a higher anomaly score. In this specific evaluation, the RGB color channels correspond to the following models:

- Source IPs: Red (LOF), Green (OCSVM), Blue (COPOD)
- Destination IPs: Red (LOF), Green (KNN), Blue (COPOD)

This process results in a dataset of 4,214 image pairs representing anomaly scores of aggregates by source and destination over Δ_T, which will be used to train and evaluate the attack pattern recognition module.

4.6.2. IP Address Positioning

In our, IP positions within the traffic representations are not fixed across images. This design choice is intentional to prevent the CNN from learning to focus on specific regions where attacked IPs are repeatedly located in the dataset, which could lead to overfitting and reduced model generalizability. By randomizing IP positions, the model is compelled to analyze the entirety of the image, thereby detecting anomalies based on the broader context rather than fixed spatial patterns.

4.6.3. Identification of Unexplained Benign Anomalies

Within the generated visual representations, we observed certain anomalies that, while flagged by our models, were confirmed as benign. Upon examining the images (Figure 10), some lines corresponding to internal IPs exhibited recurring high anomaly scores despite being benign. Initial hypotheses included server-related functions, yet further investigation did not reveal any distinguishing hardware or configuration roles.

The absence of detailed information about machine activity in the CSE-CIC-IDS2018 dataset [29] limits the interpretation of these benign anomalies. In operational environments, these anomalies would warrant deeper analysis with network administrators to understand the specific nature of these machines and adjust models accordingly. Network segmentation and asset-focused anomaly detection could also support this contextual analysis in practical deployments.

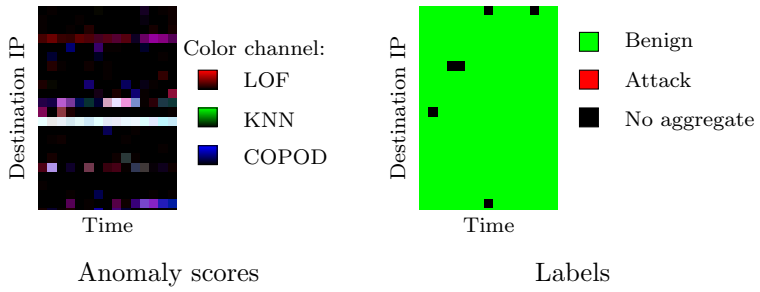

Figure 10: Unexplained benign anomalies

4.6.4. Attack Pattern Recognition Module Evaluation

The attack pattern recognition module, based on a CNN, is designed to analyze RGB images of anomaly scores and detect complex attack patterns.

Table 9: CNN results

	TPR	**FPR**	**F-score**	**Confusion matrix**			
				TP	**FP**	**FN**	**TN**
CNN Source	0.9796	0.0062	0.9905	96	2	2	322
CNN Destination	0.9796	0.0093	0.9882	96	3	2	321
CNN Combined	**0.9898**	**0.0093**	**0.9906**	**97**	**3**	**1**	**321**

Each image, representing a 30-minute traffic interval, is classified as an attack or non-attack based on the presence of an attack aggregate. This final dataset for training consists of 4,214 image pairs, each labeled for binary classification.

To train and evaluate the CNN, we split the data into three sets: 80% for training, 10% for validation, and 10% for testing. The training set is used to adjust the CNN's parameters, while the validation set monitors performance to prevent overfitting, ensuring generalizability.

The test set provides a final performance measure of the model, assessing its ability to detect attack patterns on previously unseen data. This cross-validation process ensures that the model maintains robustness beyond training data.

CNNs require tuning of numerous hyperparameters, such as filter size, number of layers, and dropout rates. We used Keras-Tuner [25] to automate this process, systematically evaluating configurations based on weighted validation accuracy (val_accuracy), which accounts for class imbalances between attack and benign periods.

Table 9 shows the F-score and confusion matrix of our CNN-based attack pattern recognition module, comparing models for source IPs, destination IPs, and combined. The combined model achieves a slightly better F-score, with three false positives and one missed attack (false negative).

An examination of the false negative (Figure 11) revealed that it corresponded to the very end of an attack, represented by a single pixel in the destination aggregates, explaining why the recognition module failed to detect it. This false negative would have minimal practical impact, as the attack had already been detected in the previous time frame and had already ended.

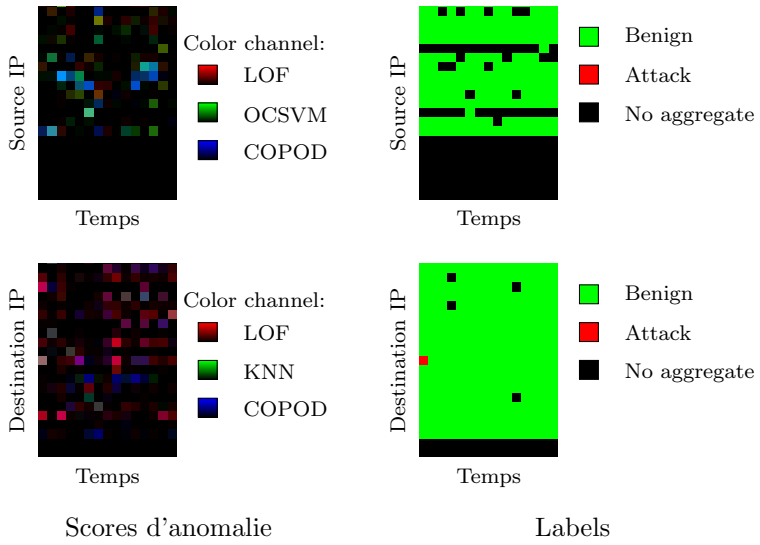

Figure 11: False negative. The upper part shows the source-based aggregates, where the attack is not visible, while the lower part presents the destination-based aggregates. A single red pixel appears at the far left of the image representing the labels of the destination-based aggregates, indicating the end of an attack. The base learners detected this attack in the destination representation, as evidenced by the presence of a light pixel on the image representing the anomaly scores.

4.7. Explainability for Enhanced Performance

Our system's goal is not to achieve the highest possible accuracy through purely statistical means, as seen in many prior works. Instead, we aim for perfect accuracy by engaging the end user of a NIDS, namely, a human analyst who must interpret the results and decide on actions if an attack on the network is suspected.

Our system is inherently designed for explainability: each successive component enables tracing back to the specific aggregate features that triggered the anomaly detection. Each alert triggers a concise visual representation of the network's state at that moment, displaying all anomaly scores across aggregates. This approach allows analysts to view the aggregates involved in the attack and trace back to the anomalous feature pairs identified by unsupervised models. This information can then form the basis of a detailed attack report for security analysts, which will ultimately be integrated into Custocy's NDR solution [1].

A core principle of our approach is that explainability is not simply an

independent goal but one that directly enhances performance. Rather than accepting the conventional trade-off between performance and explainability, we demonstrate that transparent and interpretable detection processes improve both the accuracy and effectiveness of the system. By offering clear insight into the detection mechanisms, our system enables more precise attack pattern identification, reducing the risk of overfitting or false alarms and resulting in better overall efficiency.

To illustrate how explainability allows us to achieve perfect accuracy on our dataset, we examined the image corresponding to the undetected attack in Table 9 (Figure 11). This image represented the end of an attack, which would have been detected by preceding images as the attack spanned multiple frames. In practice, this attack would thus have been detected, resulting in zero false negatives in our evaluation dataset.

Moreover, explainability makes our model more robust against emerging threats. Intuitive visualizations allow us to show anomalies even when they do not match previously known attack patterns, revealing new traffic patterns. Thus, our explainable approach not only enhances trust in the system but also contributes to adaptability and handling of varied scenarios.

Finally, to situate our method in the context of existing NIDS research, we compared its detection performance to those reported in a recent survey by Leevy and Khoshgoftaar [18], which compiles results on the same dataset (CSE-CIC-IDS2018). While several approaches—including deep neural networks and ensemble architectures—achieve similar or slightly higher accuracy scores (e.g., Atefnia and Ahmadi [5]), they typically rely on black-box models that provide little to no interpretability. To the best of our knowledge, our system is the only method in this accuracy range that natively integrates interpretability into every step of the detection process, offering both strong detection capability and transparent decision support. This positions our approach as a promising direction for real-world integration in NDR systems, where trust is essential.

5. Conclusion

In this paper, we introduced an innovative approach for network intrusion detection that prioritizes explainability, addressing a critical need in network security. Our method moves beyond post-hoc tools like SHAP or LIME by embedding interpretability directly into the detection pipeline, through a transparent ensemble-based NIDS and a CNN meta-learner. This integration

enhances transparency and gives security analysts a deeper understanding of each detection outcome.

By analyzing visual representations of anomaly scores, our system provides interpretable insights into network behavior, making it easier for analysts to identify and assess potential threats.

Our approach demonstrates that explainability can be effectively integrated into NIDS systems to improve both performance and usability. This work contributes to the development of intrusion detection solutions that are not only robust in identifying complex threats but also interpretable and actionable, bridging the gap between advanced ML capabilities and practical security needs.

Overall, our approach provides a modular and interpretable framework for unsupervised anomaly detection, combining ensemble learning with visual pattern recognition. It yields strong performance on standard benchmarks and lays the groundwork for explainable intrusion detection systems. Developed within Custocy's R&D department, this prototype aims to validate the technical feasibility and interpretability of methods intended for future integration into the company's NDR solution. Deployment-related aspects such as engineering effort or resource cost are beyond the current scope. Future work includes extending this approach to more recent datasets or simulated environments that reflect evolving attack paradigms, such as AI-generated threats. Further statistical evaluation will be necessary to consolidate the robustness of our results. Finally, assessing the practical usability of the visual outputs through feedback from security analysts remains a key direction for future research.

Acknowledgments

We gratefully acknowledge the financial support provided by Custocy and the French National Association for Research and Technology (ANRT) through the CIFRE (Industrial Agreement for Training through Research) program. We also thank our colleagues at Custocy for their support in using the Netsens tool. Furthermore, we extend our appreciation to the team responsible for the LAAS-CNRS pfcalcul platform for granting us access and providing technical support.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used ChatGPT in order to improve the readability and language of the manuscript. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

References

[1] Custocy. https://www.custocy.ai/.

[2] The MITRE Corporation. https://www.mitre.org/.

[3] Bushra A. Alahmadi, Louise Axon, and Ivan Martinovic. 99% False Positives: A Qualitative Study of SOC Analysts' Perspectives on Security Alarms. In *Proceedings of the 31st USENIX Security Symposium*, pages 2783–2800, Boston, MA, USA, 2022. USENIX Association.

[4] Fabrizio Angiulli and Clara Pizzuti. Fast outlier detection in high dimensional spaces. In *Proceedings of the 6th European Conference on Principles of Data Mining and Knowledge Discovery (PKDD 2002)*, volume 2431 of *Lecture Notes in Computer Science*, pages 15–27, Helsinki, Finland, 2002. Springer.

[5] Ramin Atefinia and Mahmood Ahmadi. Network intrusion detection using multi-architectural modular deep neural network. *The Journal of Supercomputing*, 77(4):3571–3593, April 2021.

[6] Leo Breiman. Bagging predictors. *Machine Learning*, 24(2):123–140, August 1996.

[7] Markus M. Breunig, Hans-Peter Kriegel, Raymond T. Ng, and Jörg Sander. LOF: Identifying density-based local outliers. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, pages 93–104, New York, NY, USA, May 2000. Association for Computing Machinery.

[8] Pedro Casas, Johan Mazel, and Philippe Owezarski. UNADA: Unsupervised Network Anomaly Detection Using Sub-space Outliers Ranking. In *Proceedings of the 10th IFIP Networking Conference*, volume 6640 of

Lecture Notes in Computer Science, pages 40–51, Valencia, Spain, May 2011. Springer.

[9] Juliette Dromard, Gilles Roudière, and Philippe Owezarski. Online and Scalable Unsupervised Network Anomaly Detection Method. *IEEE Transactions on Network and Service Management*, 14(1):34–47, March 2017.

[10] Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96)*, pages 226–231, Portland, OR, USA, 1996. AAAI Press.

[11] Yoav Freund and Robert E Schapire. A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting. *Journal of Computer and System Sciences*, 55(1):119–139, August 1997.

[12] Jerome H. Friedman. Greedy Function Approximation: A Gradient Boosting Machine. *The Annals of Statistics*, 29(5):1189–1232, 2001.

[13] Ibrahim Ghafir, Konstantinos G. Kyriakopoulos, Sangarapillai Lambotharan, Francisco J. Aparicio-Navarro, Basil Assadhan, Hamad Binsalleeh, and Diab M. Diab. Hidden Markov Models and Alert Correlations for the Prediction of Advanced Persistent Threats. *IEEE Access*, 7:99508–99520, 2019.

[14] Dongqi Han, Zhiliang Wang, Wenqi Chen, Ying Zhong, Su Wang, Han Zhang, Jiahai Yang, Xingang Shi, and Xia Yin. DeepAID: Interpreting and Improving Deep Learning-based Anomaly Detection in Security Applications. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 3197–3217, Virtual Event Republic of Korea, November 2021. Association for Computing Machinery.

[15] Wajih Ul Hassan, Shengjian Guo, Ding Li, Zhengzhang Chen, Kangkook Jee, Zhichun Li, and Adam Bates. NoDoze: Combatting Threat Alert Fatigue with Automated Provenance Triage. In *Proceedings of the 26th Annual Network and Distributed System Security Symposium (NDSS)*, San Diego, CA, USA, February 2019.

[16] Eric M Hutchins, Michael J Cloppert, and Rohan M Amin. Intelligence-Driven Computer Network Defense Informed by Analysis of Adversary Campaigns and Intrusion Kill Chains. page 14, 2011.

[17] Maxime Lanvin, Pierre-François Gimenez, Yufei Han, Frédéric Majorczyk, Ludovic Mé, and Éric Totel. Errors in the CICIDS2017 Dataset and the Significant Differences in Detection Performances It Makes. In *Proceedings of the 17th International Conference on Risks and Security of Internet and Systems (CRiSIS)*, pages 18–33, Sousse, Tunisia, 2023. Springer.

[18] Joffrey L. Leevy and Taghi M. Khoshgoftaar. A survey and analysis of intrusion detection models based on CSE-CIC-IDS2018 Big Data. *Journal of Big Data*, 7(1):104, November 2020.

[19] Zheng Li, Yue Zhao, Nicola Botta, Cezar Ionescu, and Xiyang Hu. COPOD: Copula-Based Outlier Detection. In *Proceedings of the 2020 IEEE International Conference on Data Mining (ICDM)*, pages 1118–1123, Sorrento, Italy, November 2020. IEEE.

[20] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation Forest. In *Proceedings of the Eighth IEEE International Conference on Data Mining (ICDM)*, pages 413–422, Pisa, Italy, December 2008. IEEE.

[21] Scott M Lundberg and Su-In Lee. A Unified Approach to Interpreting Model Predictions. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.

[22] Dan McWhorter. Mandiant Exposes APT1 — One of China's Cyber Espionage Units & Releases 3,000 Indicators. *Mandiant, February*, 2013.

[23] Sadegh M. Milajerdi, Rigel Gjomemo, Birhanu Eshete, R. Sekar, and V.N. Venkatakrishnan. HOLMES: Real-Time APT Detection through Correlation of Suspicious Information Flows. In *Proceedings of the 40th IEEE Symposium on Security and Privacy (S&P)*, pages 1137–1152, San Francisco, CA, USA, May 2019. IEEE.

[24] Yisroel Mirsky, Tomer Doitshman, Yuval Elovici, and Asaf Shabtai. Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection. In *Proceedings of the 25th Network and Distributed System Security Symposium*, San Diego, CA, 2018. Internet Society.

[25] Tom O'Malley, Elie Bursztein, James Long, François Chollet, Haifeng Jin, Luca Invernizzi, et al. KerasTuner, 2019.

[26] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake VanderPlas, Alexandre Passos, and David Cournapeau. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[27] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. "Why Should I Trust You?": Explaining the Predictions of Any Classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '16, pages 1135–1144, New York, NY, USA, August 2016. Association for Computing Machinery.

[28] Bernhard Schölkopf, Robert C Williamson, Alex Smola, John Shawe-Taylor, and John Platt. Support Vector Method for Novelty Detection. In *Advances in Neural Information Processing Systems*, volume 12, pages 582–588, Denver, CO, USA, 1999. MIT Press.

[29] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy*, pages 108–116, Funchal, Madeira, Portugal, 2018. SCITEPRESS - Science and Technology Publications.

[30] Juan Vanerio and Pedro Casas. Ensemble-learning Approaches for Network Security and Anomaly Detection. In *Proceedings of the Workshop on Big Data Analytics and Machine Learning for Data Communication Networks*, pages 1–6, Los Angeles CA USA, August 2017. Association for Computing Machinery.

[31] Abdul Waleed, Abdul Fareed Jamali, and Ammar Masood. Which open-source IDS? Snort, Suricata or Zeek. *Computer Networks*, 213:109116, August 2022.

[32] Wei Wang, Ming Zhu, Xuewen Zeng, Xiaozhou Ye, and Yiqiang Sheng. Malware traffic classification using convolutional neural network for representation learning. In *Proceedings of the 2017 International Confer-*

ence on Information Networking (ICOIN), pages 712–717, Da Nang, Vietnam, January 2017. IEEE.

[33] Feng Wei, Hongda Li, Ziming Zhao, and Hongxin Hu. {xNIDS}: Explaining Deep Learning-based Network Intrusion Detection Systems for Active Intrusion Responses. In *Proceedings of the 32nd USENIX Security Symposium (USENIX Security 23)*, pages 4337–4354, Anaheim, CA, USA, August 2023. USENIX Association.

[34] David H. Wolpert. Stacked generalization. *Neural Networks*, 5(2):241–259, January 1992.

[35] Yue Zhao, Zain Nasrullah, and Zheng Li. PyOD: A Python Toolbox for Scalable Outlier Detection. *Journal of Machine Learning Research*, 20(96):1–7, 2019.

[36] Peng Zhou, Gongyan Zhou, Dakui Wu, and Minrui Fei. Detecting multi-stage attacks using sequence-to-sequence model. *Computers & Security*, 105:102203, June 2021.